



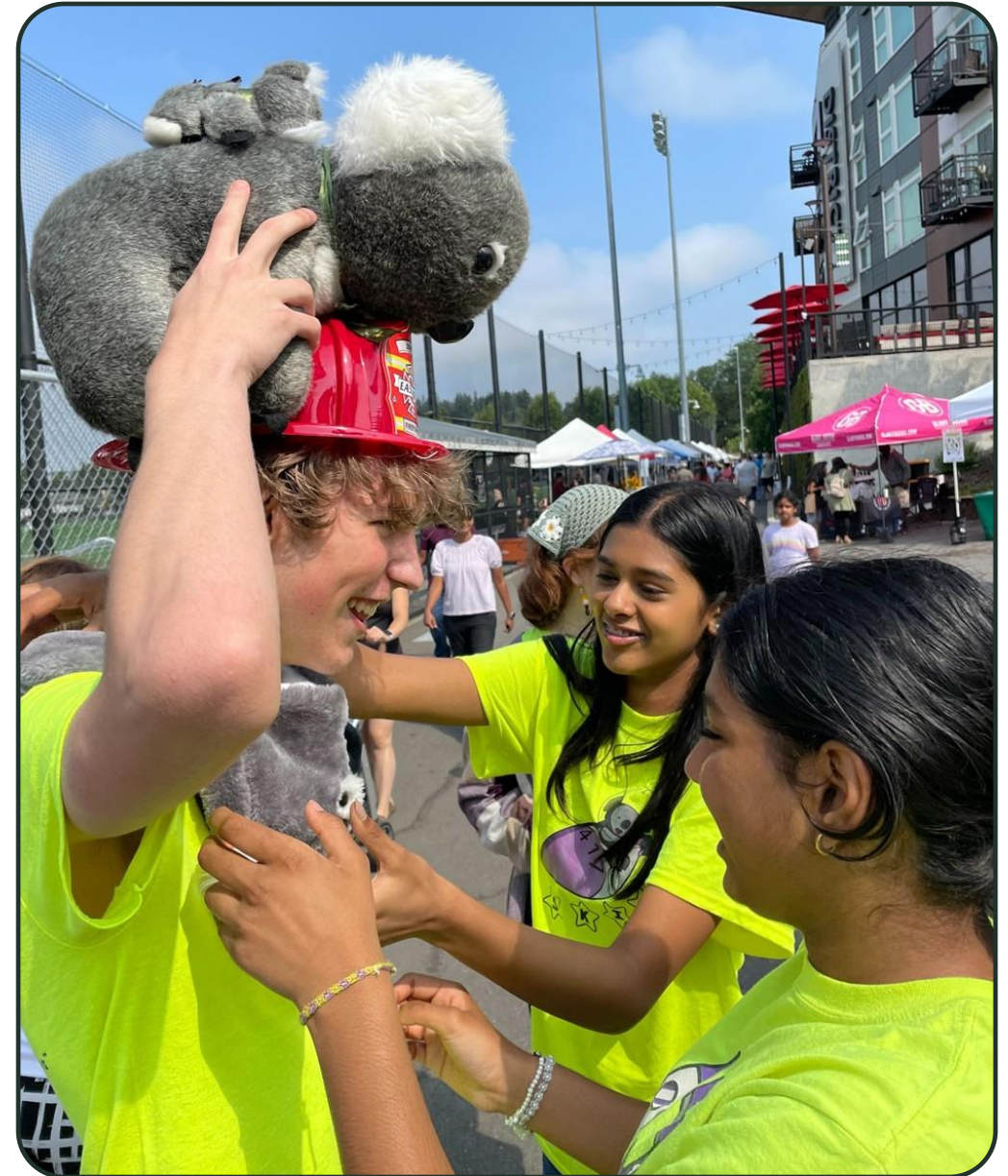
The Basics of Making Robots Move

By Eric Goossen



Who am I?

- Eric Goossen
- General nuisance
- FTC alum and former captain of 417 SKID
- Competed in FTC for two years
- Competed in Robots overall for six years
- Current student at the Paul G Allen school of computer science at UW





What am I Talking about?

- Focusing on pathing and control theory
- What types of libraries already exist to solve this problem?
- How my teams have tackled this problem over the years:
 - Dead reckoning
 - Linear Pathing
 - Curvilinear Pathing
- The library I made with my code
- A link to these slides will be at the end

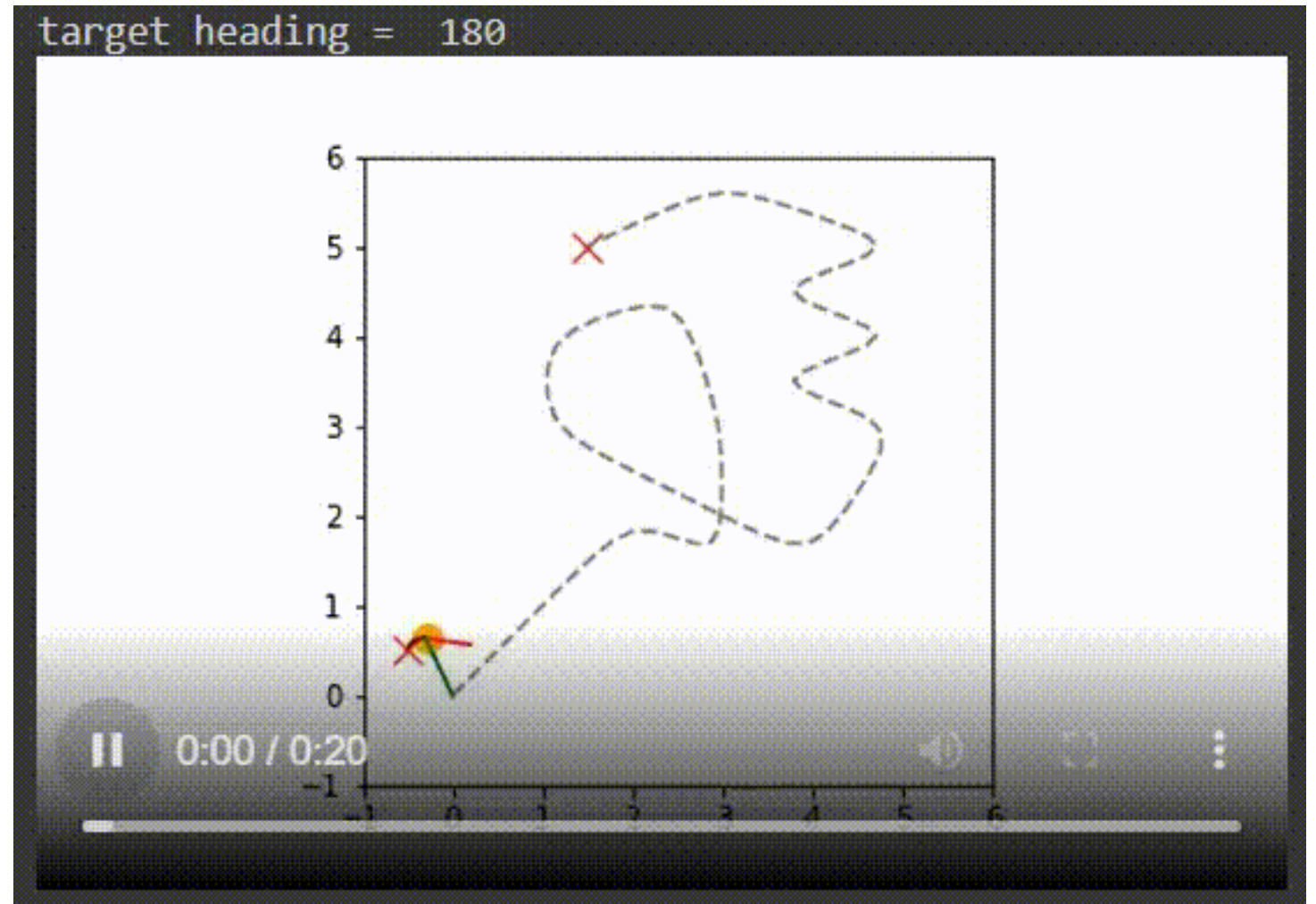


What is Pathing?

- Pathing is all about getting from point A to point B
- You can follow curved paths or just straight lines
- Can incorporate velocity control to prevent overshoot
- The building block for autonomous control
- Can be used during Tele-op to automatically move to score
- Focus on holonomic drives
- Pathing is dependent on localization
 - I'm not going to be talking about it, as it could be its own presentation
- Angular velocity is not included

Path Followers For FTC - How Not To Build Your Own

- Path followers can be intimidating to build from scratch
 - ...and take a long time
- Fortunately, solutions exist for FTC like...
 - Pure Pursuit
 - Road Runner
 - Pedro Pathing
- We'll briefly look at each of these

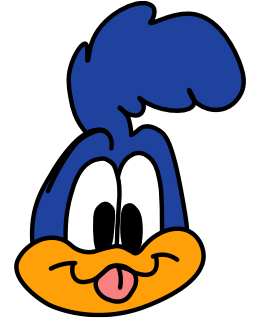


Path Followers: Pure Pursuit



- The granddaddy of pathing algorithms
 - Designed at CMU in 1992
 - At its core, it's a method for calculating the arc necessary to get a robot back onto a path
 - Purdue Sigbots has the best explanation
 - Code for FTC can be found at FTCLib (deprecated) and Tech Toolbox
- It's a great starting point for understanding the challenges of path following...
 - ...but it's bare-bones and not close to a complete solution for FTC
 - You'll be on your own for PID integration, tuning, curve support, path creation tools, etc.
- Worth learning from, but don't use it

Path Followers: Road Runner



The Good

- The most widely known pathing solution in FTC
- Complete solution with powerful feature set
 - Has support for event triggering, path visualization, logging, etc.
- Robust, automatic tuning
 - For all path followers, many constants must be tuned based on the performance characteristics of your robot
- Mature and stable codebase

The Not So Good

- No centripetal force compensation
 - Thinks it can go around any curve, no matter how sharp, at max speed
- Requires computationally expensive precomputation
 - ...so is not good for TeleOp and not good at handling unexpected disturbances
- Is conservative for speed and acceleration
 - Your robot won't run as fast as in TeleOp
- The code is arcane and poorly commented

Path Followers: Pedro Pathing



The Good

- Maxes out robot speed
- Centripetal force correction helps the robot stick to sharp turns
- Browser-based visualizer is super easy to use

The Not So Good

- New and not yet well proven
- Bunch of manual tuning required
- Code is nicely documented
- I don't have a ton of experience with this yet

Deconstruct Road Runner to Build Your Own Pathing



- All path followers require unique tuning for every robot
 - ...and writing tuning software is a pain, as is the tuning process itself
 - Road Runner's tuning is mature, automatic and easy
 - ...so why not take advantage of Road Runner's tuning support?
- Idea: build your own path follower on top of Road Runner's tuning!
 - Call Road Runner's low-level functions that use tuned parameters
- Unfortunately, Road Runner's code is a bit obtuse and poorly commented
 - ...so, we'll walk through the important bits here
- `FollowTrajectoryAction::run()` is Road Runner's workhorse routine
 - It gets called every ~20ms and is responsible for using the current time and robot position to set the drive motors appropriately
- It can be found in **`MecanumDrive.java`** in the QuickStart portion of Road Runner
 - ...so, it's a file in your package that you can easily modify or copy
- Let's walk through what Road Runner's code does...

FollowTrajectoryAction::run()

Road Runner Guts, part 1

- Here Road Runner calculates the time since the trajectory started
- It stops the robot once that value exceeds the time calculated to do the entire trajectory

```
public final class FollowTrajectoryAction implements Action {
    public boolean run(@NonNull TelemetryPacket p) {
        double t;
        if (beginTs < 0) {
            beginTs = Actions.now();
            t = 0;
        } else {
            t = Actions.now() - beginTs;
        }

        if (t >= timeTrajectory.duration) {
            leftFront.setPower(0);
            leftBack.setPower(0);
            rightBack.setPower(0);
            rightFront.setPower(0);

            return false;
        }

        Pose2dDual<Time> txWorldTarget = timeTrajectory.get(t);
        targetPoseWriter.write(new PoseMessage(txWorldTarget.value()));

        PoseVelocity2d robotVelRobot = updatePoseEstimate();

        PoseVelocity2dDual<Time> command = new HolonomicController(
            PARAMS.axialGain, PARAMS.lateralGain, PARAMS.headingGain,
            PARAMS.axialVelGain, PARAMS.lateralVelGain, PARAMS.headingVelGain
        )
            .compute(txWorldTarget, localizer.getPose(), robotVelRobot);

        driveCommandWriter.write(new DriveCommandMessage(command));

        MecanumKinematics.WheelVelocities<Time> wheelVels = kinematics.inverse(command);
        double voltage = voltageSensor.getVoltage();

        final MotorFeedforward feedforward = new MotorFeedforward(PARAMS.kS,
            PARAMS.kV / PARAMS.inPerTick, PARAMS.kA / PARAMS.inPerTick);
        double leftFrontPower = feedforward.compute(wheelVels.leftFront) / voltage;
        double leftBackPower = feedforward.compute(wheelVels.leftBack) / voltage;
        double rightBackPower = feedforward.compute(wheelVels.rightBack) / voltage;
        double rightFrontPower = feedforward.compute(wheelVels.rightFront) / voltage;
        mecanumCommandWriter.write(new MecanumCommandMessage(
            voltage, leftFrontPower, leftBackPower, rightBackPower, rightFrontPower
        ));

        leftFront.setPower(leftFrontPower);
        leftBack.setPower(leftBackPower);
        rightBack.setPower(rightBackPower);
        rightFront.setPower(rightFrontPower);
    }
}
```

FollowTrajectoryAction::run()

Road Runner guts, part 2

- `txWorldTarget` will be the robot's target pose, velocity and acceleration for this iteration
 - It uses a funky `Pose2Dual<Time>` data structure that encompasses all three of those vectors
 - Each vector is an (x, y) position and a heading
 - Road Runner pre-calculates all of these for the entire run when you create the trajectory
 - ...so, `get()` does a simple interpolated look-up
- When you're doing your own pathing, you won't call `get()`
 - Instead, you'll calculate your own position, velocity and acceleration values
 - Create your own `txWorldTarget`, like this, to feed other Road Runner routines:


```
double[] x = { xPosition, xLinearVel, xLinearAccel };
double[] y = { yPosition, yLinearVel, yLinearAccel };
double[] angular = { heading, angVel, angAccel };

Pose2dDual<Time> txWorldTarget = new Pose2dDual<>(
    new Vector2dDual<>(new DualNum<>(x), new DualNum<>(y)),
    Rotation2dDual.exp(new DualNum<>(angular)));
```
- Units should be inches/s, radians/s, and in field-relative coordinates
- `updatePoseEstimate()` gets the robot's current field-relative pose and robot-relative velocity from odometry
- Then `HolonomicController::compute()` gets called
 - It computes the new *command* velocity needed to go from the current position and velocity to the target for the next iteration
 - Let's look into this routine next...

```
public final class FollowTrajectoryAction implements Action {
    public boolean run(@NonNull TelemetryPacket p) {
        double t;
        if (beginTs < 0) {
            beginTs = Actions.now();
            t = 0;
        } else {
            t = Actions.now() - beginTs;
        }

        if (t >= timeTrajectory.duration) {
            leftFront.setPower(0);
            leftBack.setPower(0);
            rightBack.setPower(0);
            rightFront.setPower(0);

            return false;
        }

        Pose2dDual<Time> txWorldTarget = timeTrajectory.get(t);
        targetPoseWriter.write(new PoseMessage(txWorldTarget.value()));

        PoseVelocity2d robotVelRobot = updatePoseEstimate();

        PoseVelocity2dDual<Time> command = new HolonomicController(
            PARAMS.axialGain, PARAMS.lateralGain, PARAMS.headingGain,
            PARAMS.axialVelGain, PARAMS.lateralVelGain, PARAMS.headingVelGain
        )
            .compute(txWorldTarget, localizer.getPose(), robotVelRobot);

        driveCommandWriter.write(new DriveCommandMessage(command));

        MecanumKinematics.WheelVelocities<Time> wheelVels = kinematics.inverse(command);
        double voltage = voltageSensor.getVoltage();

        final MotorFeedforward feedforward = new MotorFeedforward(PARAMS.kS,
            PARAMS.kV / PARAMS.inPerTick, PARAMS.kA / PARAMS.inPerTick);
        double leftFrontPower = feedforward.compute(wheelVels.leftFront) / voltage;
        double leftBackPower = feedforward.compute(wheelVels.leftBack) / voltage;
        double rightBackPower = feedforward.compute(wheelVels.rightBack) / voltage;
        double rightFrontPower = feedforward.compute(wheelVels.rightFront) / voltage;
        mecanumCommandWriter.write(new MecanumCommandMessage(
            voltage, leftFrontPower, leftBackPower, rightBackPower, rightFrontPower
        ));

        leftFront.setPower(leftFrontPower);
        leftBack.setPower(leftBackPower);
        rightBack.setPower(rightBackPower);
        rightFront.setPower(rightFrontPower);
    }
}
```

HolonomicController::compute()

Road Runner library routine

- This is a Kotlin library function that returns a robot-relative *command* velocity vector
 - It's in the Road Runner library itself
 - If you need to modify its behavior, copy the code (may need to rewrite in Java)
- This routine first converts the target field-relative velocity to be robot-relative `targetVelTarget`
- It then essentially applies a PID to correct error when the robot deviates from the expected position
 - This is the only place that Road Runner uses the pose and velocity returned from odometry
- During TeleOp, you may want to path with relative motion even if the odometry pose is inaccurate
 - Just pass in zeroes for all gains

```
class HolonomicController(  
    val axialPosGain: Double,  
    val lateralPosGain: Double,  
    val headingGain: Double,  
    val axialVelGain: Double,  
    val lateralVelGain: Double,  
    val headingVelGain: Double,  
) {  
    /**  
     * Computes the velocity and acceleration command. The frame `Target` is the reference  
     * robot, and the frame `Actual` is the measured, physical robot.  
     *  
     * @return velocity command in the actual frame  
     */  
    fun compute(  
        targetPose: Pose2dDual<Time>,  
        actualPose: Pose2d,  
        actualVelActual: PoseVelocity2d,  
    ): PoseVelocity2dDual<Time> {  
        val targetVelWorld = targetPose.velocity()  
        val txTargetWorld = Pose2dDual.constant<Time>(targetPose.value().inverse(), 2)  
        val targetVelTarget = txTargetWorld * targetVelWorld  
  
        val velErrorActual = targetVelTarget.value() - actualVelActual  
  
        val error = targetPose.value().minusExp(actualPose)  
        return targetVelTarget +  
            PoseVelocity2d(  
                Vector2d(  
                    axialPosGain * error.position.x,  
                    lateralPosGain * error.position.y,  
                ),  
                headingGain * error.heading.log(),  
            ) +  
            PoseVelocity2d(  
                Vector2d(  
                    axialVelGain * velErrorActual.linearVel.x,  
                    lateralVelGain * velErrorActual.linearVel.y,  
                ),  
                headingVelGain * velErrorActual.angVel,  
            )  
    }  
}
```

FollowTrajectoryAction::run()

Road Runner guts, part 3

- The resulting *command* velocity vector is logged for posterity
 - ...and then that vector is converted to individual wheel velocities using inverse kinematics
- It uses its Feed-Forward approximation to convert each velocity to the amount of voltage each motor needs
 - Those results are also logged
 - ...and then fed to the motors
- And it's basically done for this iteration!

```
public final class FollowTrajectoryAction implements Action {
    public boolean run(@NonNull TelemetryPacket p) {
        double t;
        if (beginTs < 0) {
            beginTs = Actions.now();
            t = 0;
        } else {
            t = Actions.now() - beginTs;
        }

        if (t >= timeTrajectory.duration) {
            leftFront.setPower(0);
            leftBack.setPower(0);
            rightBack.setPower(0);
            rightFront.setPower(0);

            return false;
        }

        Pose2dDual<Time> txWorldTarget = timeTrajectory.get(t);
        targetPoseWriter.write(new PoseMessage(txWorldTarget.value()));

        PoseVelocity2d robotVelRobot = updatePoseEstimate();

        PoseVelocity2dDual<Time> command = new HolonomicController(
            PARAMS.axialGain, PARAMS.lateralGain, PARAMS.headingGain,
            PARAMS.axialVelGain, PARAMS.lateralVelGain, PARAMS.headingVelGain
        )
            .compute(txWorldTarget, localizer.getPose(), robotVelRobot);

        driveCommandWriter.write(new DriveCommandMessage(command));

        MecanumKinematics.WheelVelocities<Time> wheelVels = kinematics.inverse(command);
        double voltage = voltageSensor.getVoltage();

        final MotorFeedforward feedforward = new MotorFeedforward(PARAMS.kS,
            PARAMS.kV / PARAMS.inPerTick, PARAMS.kA / PARAMS.inPerTick);
        double leftFrontPower = feedforward.compute(wheelVels.leftFront) / voltage;
        double leftBackPower = feedforward.compute(wheelVels.leftBack) / voltage;
        double rightBackPower = feedforward.compute(wheelVels.rightBack) / voltage;
        double rightFrontPower = feedforward.compute(wheelVels.rightFront) / voltage;
        mecanumCommandWriter.write(new MecanumCommandMessage(
            voltage, leftFrontPower, leftBackPower, rightBackPower, rightFrontPower
        ));

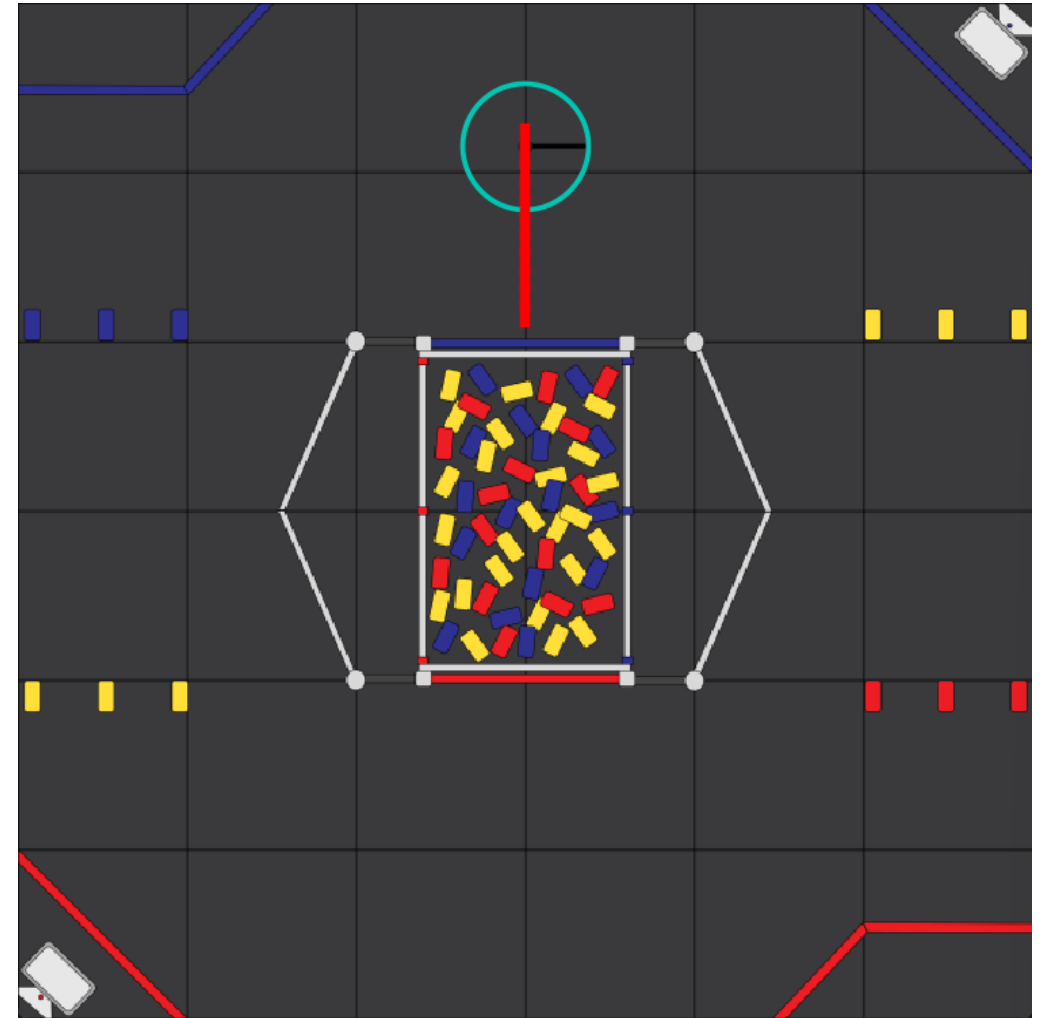
        leftFront.setPower(leftFrontPower);
        leftBack.setPower(leftBackPower);
        rightBack.setPower(rightBackPower);
        rightFront.setPower(rightFrontPower);
    }
}
```



Dead Reckoning

Simplistic by Nature

- Most basic type of pathing
- Apply a predetermined velocity over a set distance
- Come to a complete stop then apply a set velocity over a predetermined angle to turn
- Starting point to get into more advanced algorithms
- Commonly used in FLL
- Its inaccuracy and fiddliness to tune prevent it from succeeding in FTC





Dead Reckoning Summary

Pros

- Easy to write
- Simplistic and easy to debug

Cons

- No velocity smoothing can cause sensor inaccuracy
- No overshoot prevention allows differences between runs
- Predefined by the user
- Difficult to tune
- Can't adapt to changes in the environment
- Must fully stop to turn



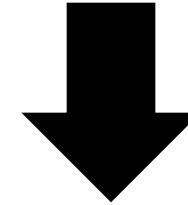
Linear Pathing



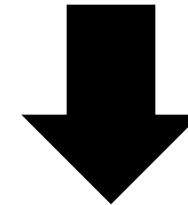
Linear Pathing

- Step up from dead reckoning
- Incorporates smooth acceleration and deceleration
- User inputs a line rather than a distance
- Allows for dynamic start and end points
- Allows for limited real time adaptation to the environment

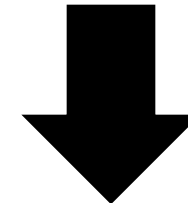
Velocity Profiling



Directional Control



PIDF



Inverse Kinematics

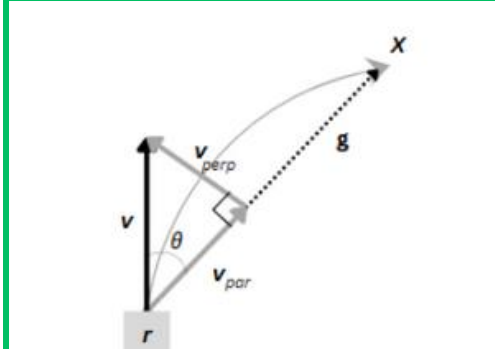
Velocity Profiling

- Smoothly accelerate over time until we hit max speed
- Cap the speed at max
- Smoothly decelerate based on the remaining distance
- This allows the robot to maintain the highest speed for the longest time possible before slowing down
- The deceleration constant must be smaller than the acceleration



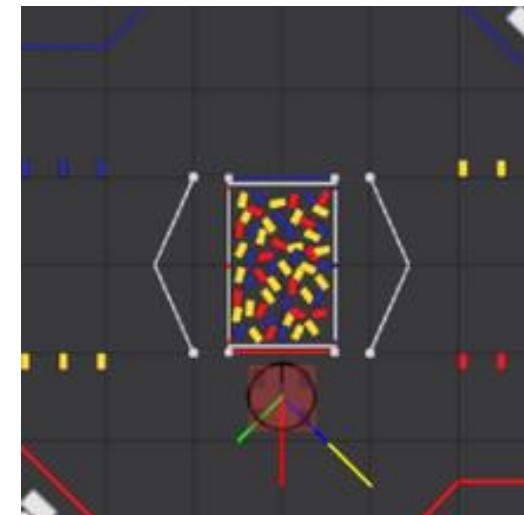
Directional Control

- Splits velocity into transverse and convergent
 - Transverse is the negative tangential plus the normal velocity
 - Convergent is the positive tangential velocity
- Prioritizes applying acceleration to bring the transverse velocity to zero
- Accelerates convergent velocity towards the velocity goal from the velocity profiling with the remaining acceleration
- Allows initial velocities to be canceled out before heading to the goal
- Tracks where it thinks the robot should be based on the velocities



The diagram shows a velocity vector $\vec{v}$ originating from a point r . It is decomposed into a parallel component $\vec{v}_{par}$ and a perpendicular component $\vec{v}_{perp}$ relative to a dashed line representing the goal direction $\vec{g}$. The angle between $\vec{v}$ and $\vec{g}$ is θ . A right-angle symbol indicates the perpendicularity of $\vec{v}_{perp}$ to $\vec{g}$.

$$v_{perp} = \text{sgn}(\theta) \cdot \max\left(\|\vec{v}\| \cdot \sin|\theta| + d \cdot \Delta t, 0\right)$$
$$v_{par} = \min\left(\|\vec{v}\| \cdot \cos\theta + a \cdot \Delta t, v_{max}, \sqrt{-2 \cdot d \cdot \|\vec{g}\|}\right)$$
$$\vec{v}_{new} = v_{par} \cdot \frac{\vec{g}}{\|\vec{g}\|} + v_{perp} \cdot \frac{\vec{g}_{\perp}}{\|\vec{g}_{\perp}\|}$$



An early unit test of the pathing code. Each line is a vector used in our velocity calculations.

PIDF

- Corrects the velocity of the robot based on the difference between the position the controller believes the robot is at and the actual position from the sensors
- Can compensate for asymmetrical hardware, like a wheel rubbing
- PID
 - Proportional: raw difference between the target position and sensor position
 - Integral: sum of proportional times the loop time
 - Derivative: Proportional delta divided by loop time
- Feedforward:
 - Velocity: target robot velocity
 - Acceleration: target robot acceleration
- PIDF is the sum of all these factors multiplied by individual tuned constants



Robot Space Conversion and Inverse Kinematics

- Takes the field relative velocities and turns them into robot relative
 - Field relative: 0 degrees is along the y-axis of the field
 - Robot relative: 0 degrees is in the direction the robot is facing
- Converts from velocities into individual wheel speeds
 - Lateral must be multiplied by a tuned constant for mecanum drives



Linear Summary

Improvements

- Smooth velocities increase sensor accuracy
- PIDF prevents overshoot, allowing the robot to consistently stop at the same spot every time
- Paths can be generated and adjusted dynamically
 - Allows for use during Tele-op without having to stop
- You only have to tune acceleration and PIDF constants once to work with all paths

Challenges

- Must stop to turn
- Requires inputting each path vertex to work
- Obstacle avoidance is impossible
- Initial velocity will cause it to move off target path

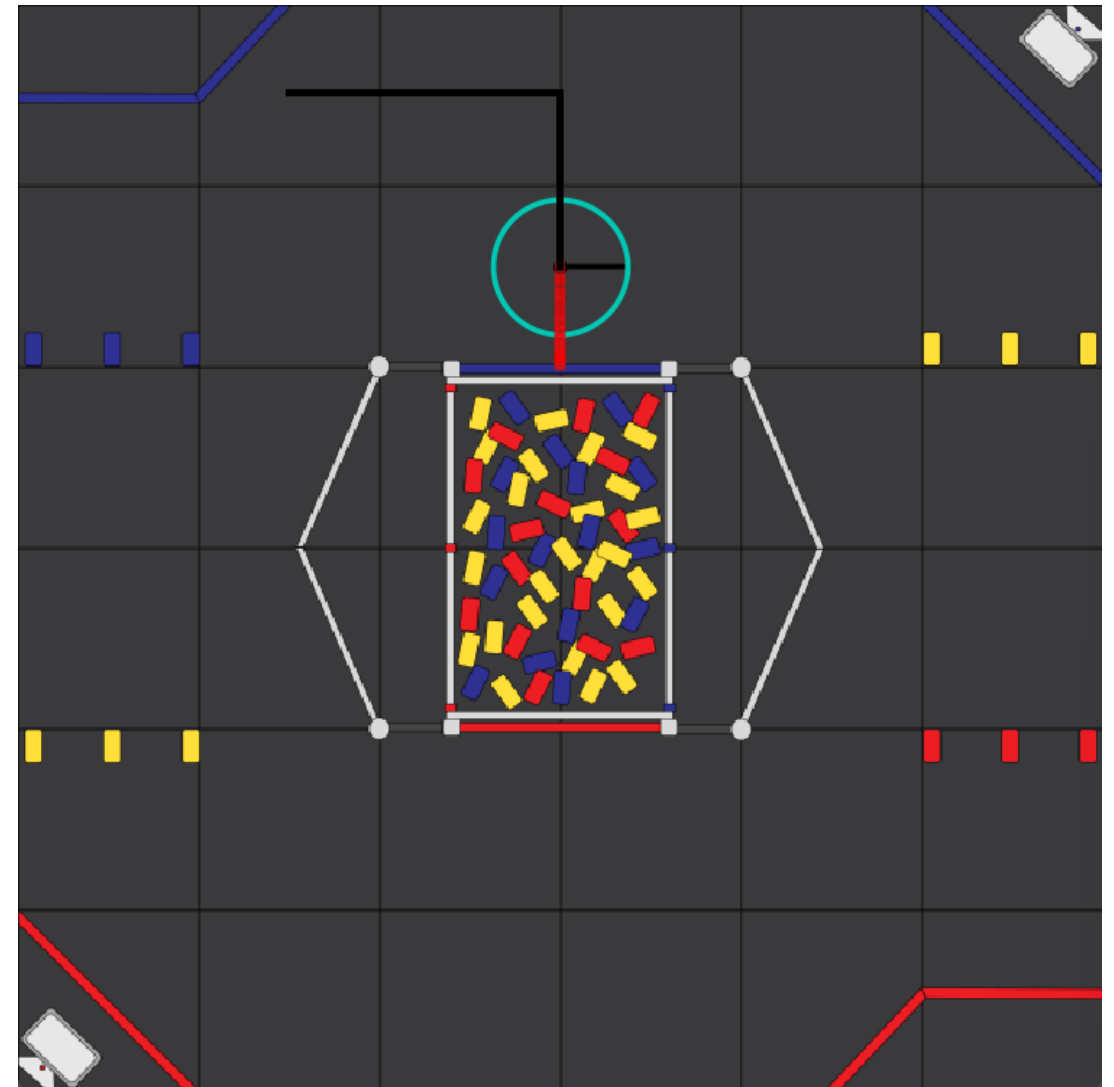


Curvilinear Pathing



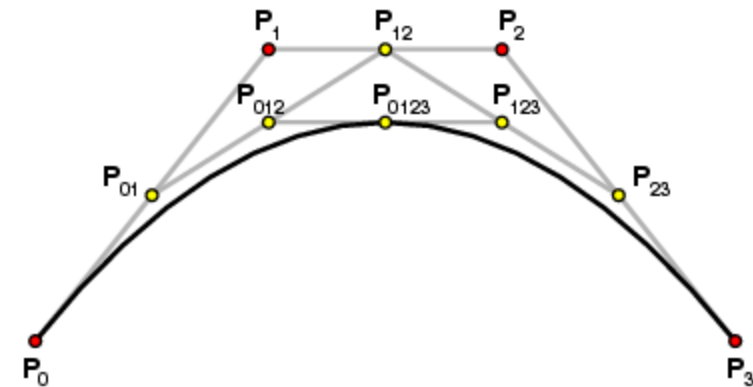
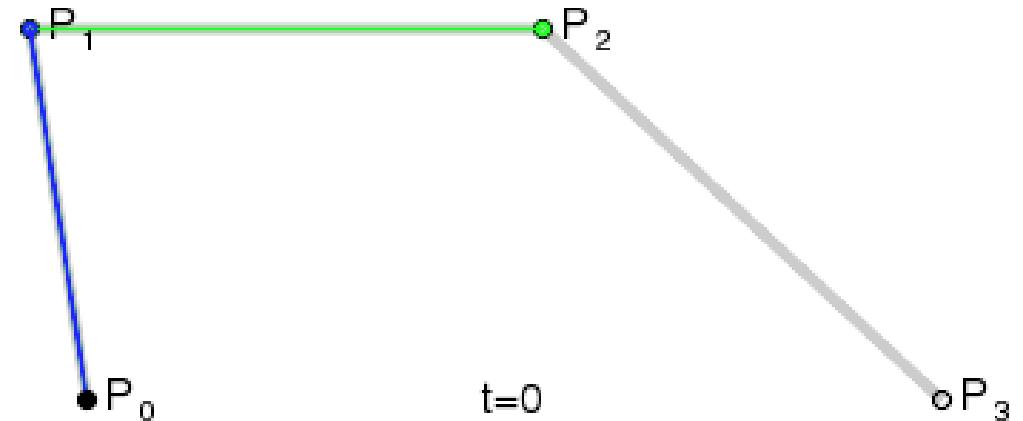
Following Consecutive Lines

- We will build off our linear path follower
 - It already decelerates the transverse velocity before accelerating towards the goal
- All we need to know is when to switch lines
- We'll calculate the distance it will take to decelerate the transverse once following the next line
 - Add an adjustment factor, because the current velocity going towards the next goal will shift the intersection
- Switch from following the current line to following the next line once that distance is reached



Bezier Curve

- Bezier curves describes a continuous differentiable path across a 2D plane
- A Bezier curve is a parametric function
 - Parametric functions are defined as x and y in terms of t, the percent of the curve that has been completed
- We focus on cubic Bezier Curves, because they are the easiest to work with
- Cubic Bezier Curves can be defined with four control points
 - The start and end points of the curve, and two additional points that shape the path between



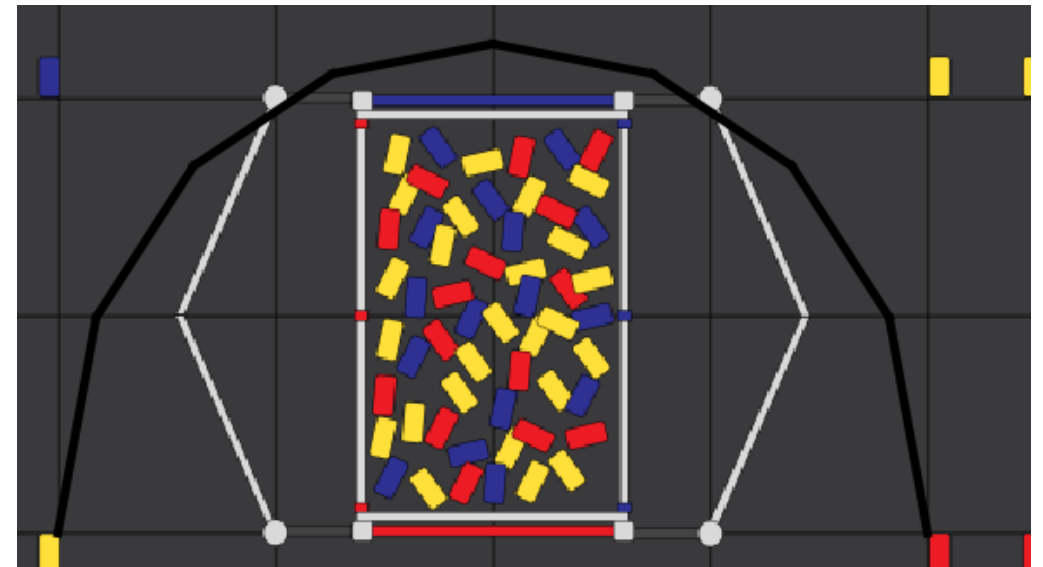
$$\mathbf{B}(t) = (1-t)^3\mathbf{P}_0 + 3(1-t)^2t\mathbf{P}_1 + 3(1-t)t^2\mathbf{P}_2 + t^3\mathbf{P}_3, \quad 0 \leq t \leq 1.$$

Equation for a Cubic Bezier

Bezier Flattening

- Turns the Bezier function into a set of usable Line segments
- Don't create a set of equal length segments, use longer segments for straighter parts of the curve
 - This allows the approximation to be adaptive to the precision demanded by the curve
- Algorithm:
 - Take the start and the end points of the curve
 - Pick a point on the curve halfway between them
 - If the lines formed by those three points are close enough to the curve, finish
 - If not, repeat but with the start and the middle point
 - Repeat until each set of three points are close enough to the curve

```
//Turns an bezier curve into an array of smaller bezier curves that resemble linear vectors.  
//epsilon = the acceptable difference between a bezier curve and a linear vector in inches.  
private ArrayList<Vector> flatten(Vector[] p, ArrayList<Vector> resultPoints, double epsilon) {  
    Vector[] l = findMidPoint(p);  
    Vector[] q = findMidPoint(l);  
    Vector c = findMidPoint(q)[0];  
  
    double error = Math.max(Math.hypot(2 * p[1].x - p[2].x - p[0].x, 2 * p[1].y - p[2].y - p[0].y),  
        Math.hypot(2 * p[2].x - p[3].x - p[1].x, 2 * p[2].y - p[3].y - p[1].y));  
  
    if (error < epsilon) {  
        resultPoints.add(p[3]);  
    } else {  
        resultPoints = flatten(new Vector[] {p[0], l[0], q[0], c}, resultPoints, epsilon);  
        resultPoints = flatten(new Vector[] {c, q[1], l[2], p[3]}, resultPoints, epsilon);  
    }  
  
    return resultPoints;  
}
```



Velocity Capping

- Local velocity cap
 - The inverse of the calculation we used earlier to calculate when to switch segments will give us the maximum velocity we can be traveling at
 - Feed this into the velocity profiler from earlier as the max speed
 - Can still build up too much velocity over multiple segments to be able to slow down to the local cap
- Lookahead velocity cap
 - There is no way to compute the exact max velocity we need to ensure each local cap can be hit
 - Instead, we can approximate it using the curvature n distance ahead and behind the robot





Curvilinear Summary

Improvements

- Curves make it possible to turn while without fully decelerating
- Can make larger movements without requiring tuning separate vertex locations
- Makes it possible to move around obstacles that are in the path, like last year's submersible
- Allows the created path to be adjusted based on the current velocity of the robot

Challenges

- It is possible for it to overshoot
- Requires tuning of values for PIDF, acceleration, line approximation, and lookahead distance



SKIDADDLE

Space Koalas In Disguise's Autonomous Dynamic Direction & Localization Engine



What is SKIDADDLE

- SKIDADDLE is the library I'm building out of the pathing algorithm I wrote for my team
- It uses the steps we just went over to do dynamic pathing during autonomous and tele-op
- I highly encourage building your own algorithm
- ...but without programming experience, starting on a massive project like this right away can be overwhelming and lead to poor results
- SKIDADDLE is open source, so you can feel free to use it directly, modify it, or copy code for your own use
- SKIDADDLE is not ready to publish and has not been tested in actual competition, but will be published next week.





Why Use SKIDADDLE

- Has full PIDF support, rather than Road Runner proportional PID controller
- Is dynamic, so it can be used during tele-op
- Includes compensation for centripetal force, allowing it to compensate for sharp curves
- Uses physics models to ensure the robot is always going as fast as possible, while maintaining the path
- Is written in Java and is much easier to for new programmers to understand than road runner
- Includes built in path creation tools



What I Want to Add

- Tuners to help with acceleration and deceleration
- Lambda functions to allow setting your own custom orientation and motor control functions
- An integrated debug bridge like FTC Dashboard
- Dynamic Bezier Curve creation
- Fusing between controller and pathing velocity inputs



In Conclusion

- Next steps I recommend:
 - If you're willing to push through early-stage development for something powerful, use SKIDDADLE
 - If you want something more common and battle proven, go [Road Runner](#)
 - If you want something in between, look into [Pedro Pathing](#)
- Take aways:
 - Go out and innovate on your own code. Don't just use the same libraries everyone else does
 - Don't be scared to try and dig through code to understand how someone else solved the same problems you're facing
 - Check out SKIDADDLE



THANK YOU!

Link to the slides: <http://github.com/SwerveRobotics/shared>

Link to SKIDADDLE: <http://github.com/SwerveRobotics/SKIDADDLE>

